



Context I

Tutoría - Semana 6

{desafío}
latam_



¿Cómo compartimos
información entre
componentes?



Autoaprendizaje

Recursos asincrónicos

¿Revisaste los recursos de la semana?

- Guía - Context API
- Hito 6 - Pizzería Mamma Mía



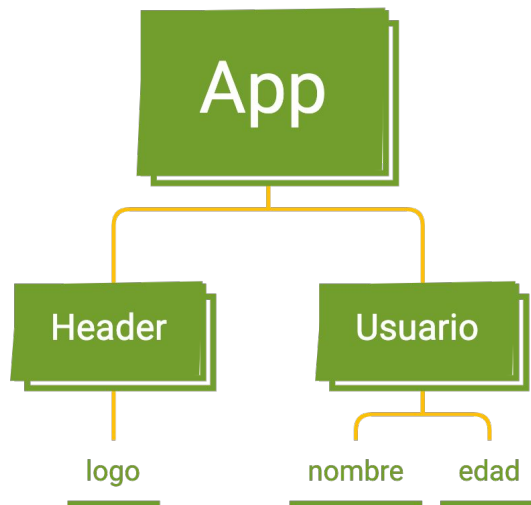
Estados y múltiples componentes

Introducción

Cada componente tiene sus propios estados, estos no se comparten ni podemos acceder directamente a ellos desde otro componente.

Es por eso que decimos << **los estados son locales** >>.

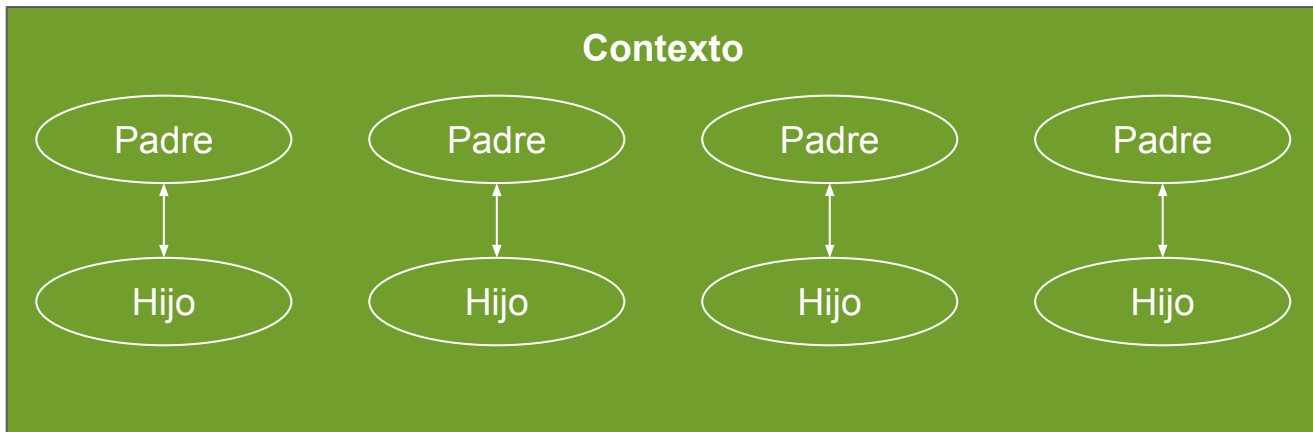
- En este diagrama tenemos el componente App, Header y Usuario.
- Header no puede cambiar el estado nombre o el estado edad de un Usuario, por lo menos no directamente.



Estados y múltiples componentes

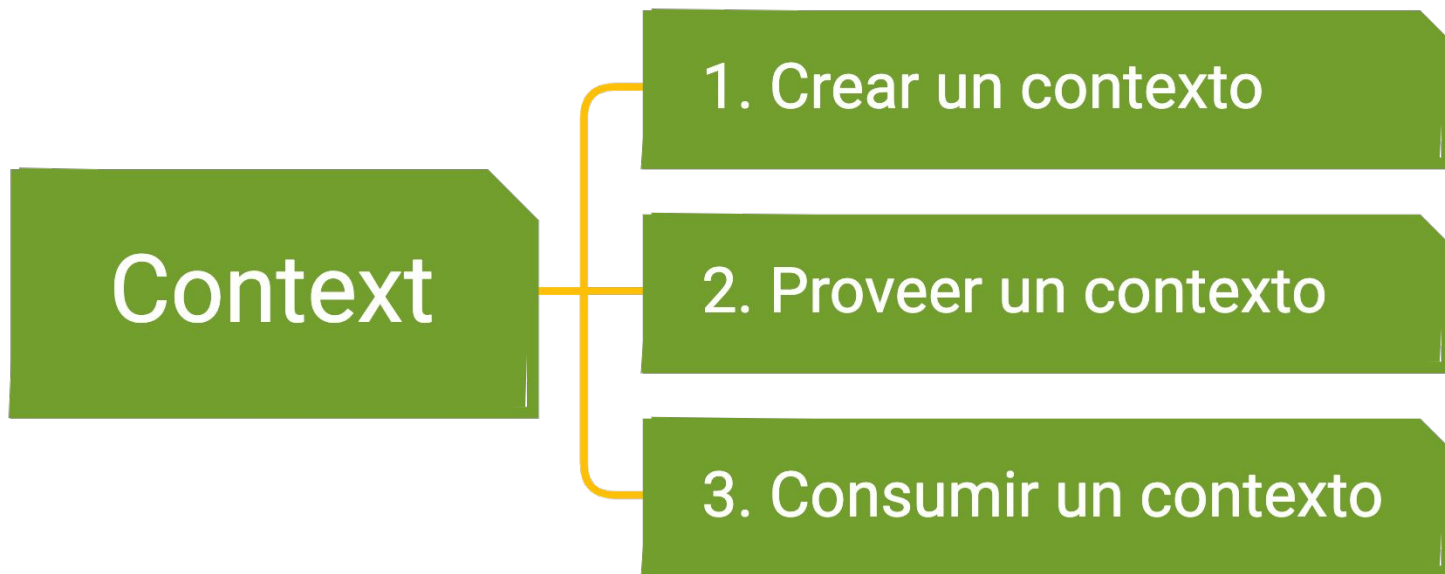
Introducción a Context

En React, podemos ocupar Context para crear un estado global o un estado de mayor alcance que un estado local, para compartir estados entre múltiples o incluso todos los componentes.



Contexto

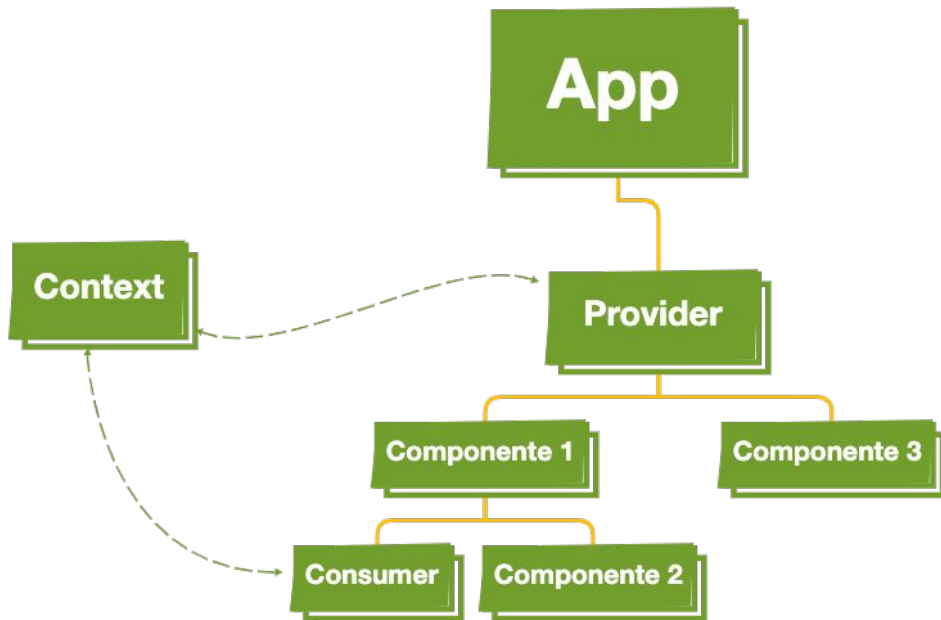
Pasos importantes de la implementación de un contexto



Contexto

Una aplicación con Context

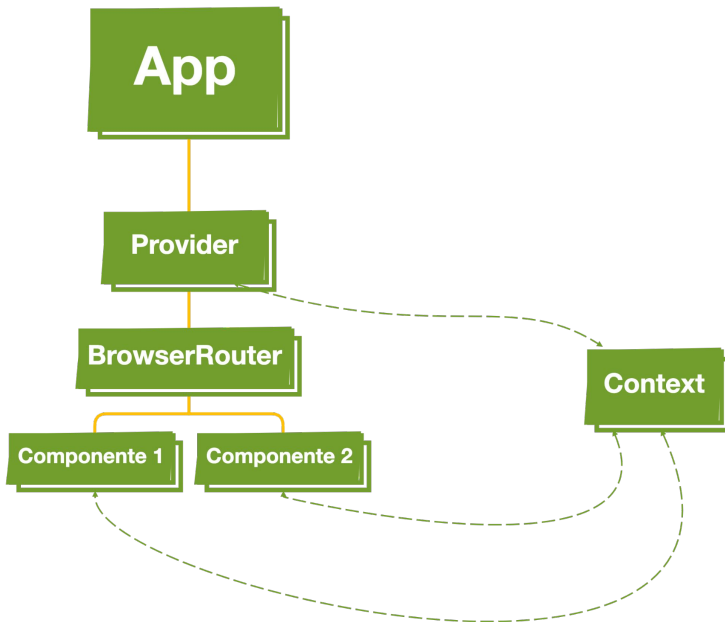
1. Cada **Context** lo debemos crear como un módulo independiente, que importaremos donde lo necesitemos.
2. Envolveremos todos los componentes que queremos mantener actualizados dentro del componente **Provider**.
3. Cualquier componente puede ser un **Consumer**. Cuando se modifique el valor de un contexto, se volverá a actualizar el consumer.



Contexto

Contexto y rutas

- Trabajar con rutas y contextos no es más complejo y se repiten los mismos pasos.
- La componente Provider debe envolver el componente Browser Router.

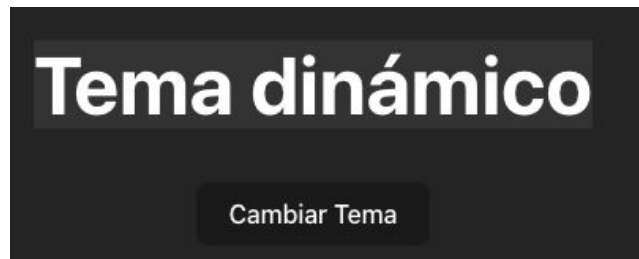


Demostración: "Modificar tema de color"



Modificar tema de color

En este ejercicio seguiremos aprendiendo sobre el contexto en React para gestionar el estado global y aplicarlo a una funcionalidad práctica: ***cambiar dinámicamente el tema de una aplicación.***



Modificar tema de color

Comenzamos creando un proyecto, en nuestro caso lo llamaremos theme-context.

Una vez creado, limpiaremos las importaciones innecesarias como los assets y creamos la siguiente estructura de carpetas:

```
theme-context/  
|-- src/  
| |-- components/  
| | |-- Header.jsx  
| | |-- ThemeToggle.jsx  
| |-- context/  
| | |-- ThemeProvider.jsx  
| |-- styles/  
| | |-- DarkTheme.css  
| | |-- LightTheme.css  
| |-- App.jsx  
|-- ...
```



Modificar tema de color

Con la estructura de carpetas ya creada, comenzaremos creando el contexto de la aplicación dentro del archivo **ThemeProvider.jsx**

Acá establecemos un contexto para gestionar el estado global del tema del componente.

- Utilizamos `useState` para mantener el estado actual del tema (light o dark)
- Utiliza el componente **ThemeContext.Provider** para envolver toda la aplicación y proveer el contexto a los componentes hijos.
- Pasamos el estado actual del tema y la función `toggleTheme` como valores del contexto.
- Creamos una función que cambiará entre los temas.



Modificar tema de color

```
import { createContext, useState, useContext } from "react";

export const ThemeContext = createContext();

const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {
    setTheme((prevTheme) => (prevTheme === "light" ? "dark" : "light"));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

export const useTheme = () => {
  return useContext(ThemeContext);
};

export default ThemeProvider;
```



Modificar tema de color

El paso siguiente es crear el encabezado de la aplicación e importamos el contexto aplicando estilos específicos para cada tema:

- Utilizamos **useTheme** para obtener el estado actual del tema desde el contexto.
- Modificamos la clase del encabezado según el tema actual para aplicar estilos específicos.
- También importamos las hojas de estilos creadas, cuyo estilo lo aplicaremos en el siguiente paso.



Modificar tema de color

```
import React from "react";
import { useTheme } from "../context/ThemeProvider";
import "../styles/DarkTheme.css";
import "../styles/LightTheme.css";

const Header = () => {
  const { theme } = useTheme();

  return (
    <header className={theme === "light" ? "light-header" : "dark-header"}>
      <h1>Tema dinámico</h1>
    </header>
  );
};

export default Header;
```



Modificar tema de color

En cada archivo CSS, definimos los estilos específicos para los temas oscuro y claro respectivamente.

```
/* DarkTheme.css */  
.dark-header {  
  background-color: #333;  
  color: #fff;  
}
```

```
/* LightTheme.css */  
.light-header {  
  background-color: #fff;  
  color: #333;  
}
```



Modificar tema de color

Ahora implementaremos un botón que permitirá cambiar el tema de la aplicación:

- En el archivo **ThemeToggle.jsx**, importa el contexto **ThemeProvider**.
- Utiliza `useTheme` para obtener la función **toggleTheme** del contexto.
- Implementamos un botón que, al hacer clic, llame a la función **toggleTheme**.

Este componente proporcionará a los usuarios una forma interactiva de cambiar entre los temas definidos.



Modificar tema de color

```
import React from "react";
import { useTheme } from "../context/ThemeProvider";

const ThemeToggle = () => {
  const { toggleTheme } = useTheme();

  return <button onClick={toggleTheme}>Cambiar Tema</button>;
};

export default ThemeToggle;
```



Modificar tema de color

Finalmente, vamos a integrar todos los componentes y contextos para formar la aplicación completa:

- Importa **ThemeProvider**, **Header**, y **ThemeToggle** en el archivo **App.jsx**.
- Dentro del componente App, coloca los componentes **Header** y **ThemeToggle** envueltos en el **ThemeProvider**.



Modificar tema de color

```
import './App.css';
import Header from './components/Header';
import ThemeToggle from './components/ThemeToggle';
import ThemeProvider from './context/ThemeProvider';

function App() {
  return (
    <>
      <ThemeProvider>
        <Header />
        <ThemeToggle />
      </ThemeProvider>
    </>
  );
}

export default App;
```



¿Existen dudas respecto
al funcionamiento de la
aplicación?





Próxima sesión...

- *React Router II*

{desafío}
latam_

*Academia de
talentos digitales*

